



Debug the Events

One sprite never moves — fix the event

Name: _____

Date: _____

★ I can find why a script never starts and fix its event so it runs.

Pixel is stuck!

Both sprites are SUPPOSED to move. The green flag starts Bit, and Bit broadcasts a message to start Pixel. But when you run it, Pixel never moves! Something is wrong with the events. Read both scripts, find why Pixel never starts, and fix it.

The number path



Bit's script

when green flag clicked

forward 2

broadcast 'go'

Pixel's script

when I receive 'jump'

back 2

1 Run it — what goes wrong?

Click the flag. Where does Bit land? Does Pixel move at all? Circle YES or NO. What number is Pixel on?

2 Find the bug

Look at the messages. Bit broadcasts ' ____ '. Pixel waits for ' ____ '. Why does Pixel never start? Write the reason.

3 Fix it

Change ONE message so they match. Write Pixel's fixed hat (or Bit's fixed broadcast). After the fix, where does Pixel land?

Big idea

A 'when I receive' script only starts if the broadcast message is EXACTLY the one it waits for. Here Bit sent 'go' but Pixel waited for 'jump', so Pixel never started. Make the two messages match and Pixel runs.



Debug the Events

Quick facilitation notes & answer key

What this worksheet teaches

The challenge sheet: students debug an event bug. Pixel never moves because the message it waits for ("jump") does not match the message Bit sends ("go"), so its event never happens. Students find the mismatch and fix it by making the two messages the same. No coding experience needed.

Before you start (no computer needed)

No computer needed — paper and a pencil. You read the prompts aloud. Optional: two counters on a number line 0 to 5 (green Bit, purple Pixel), and a card that says "go" you can hand over to act out sending a message.

How to lead it (about 20–25 minutes)

1. Show them (you do). Click the flag: Bit moves, but Pixel stays home — its message never arrived.
2. Do one together. Exercise 1 — Bit lands on 2, Pixel does not move. Exercise 2 — Bit sends "go" but Pixel waits for "jump" — they do not match.
3. Let them try. Students change one message so they match, then run Pixel (Ex 3).

Answer key (these are the verified correct answers)

Exercise 1 — Bit: 0, forward 2 → 2. Pixel does NOT move (NO); it stays on its home number 5.

Exercise 2 — Bit sends "go"; Pixel waits for "jump". The messages do not match, so Pixel's event never happens.

Exercise 3 — fix: change Pixel's hat to "when I receive go" (or change Bit's message to "jump"). With matching messages, Pixel runs: 5, back 2 → 3. Either fix works as long as both words end up the same; then Bit 2 and Pixel 3.

If students get stuck — and ways to adjust

Watch for: changing the moves instead of the message. The bug is the mismatched word, not the steps.

Make it easier: write the two message words next to each other and make them match.

Make it harder: ask which other word they could use, as long as both match.

Ontario curriculum link (official wording)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential and concurrent events.

In plain terms: students write and run instructions that are started by an event, like clicking the green flag or getting a message.

C3.2 — read and alter existing code, including code that involves sequential and concurrent events, and describe how changes to the code affect the outcomes.

In plain terms: students read instructions, change what starts them or one move, and explain what the change did.

You'll know it worked when

The child finds the mismatched messages and makes them the same so Pixel runs.